QuickBite

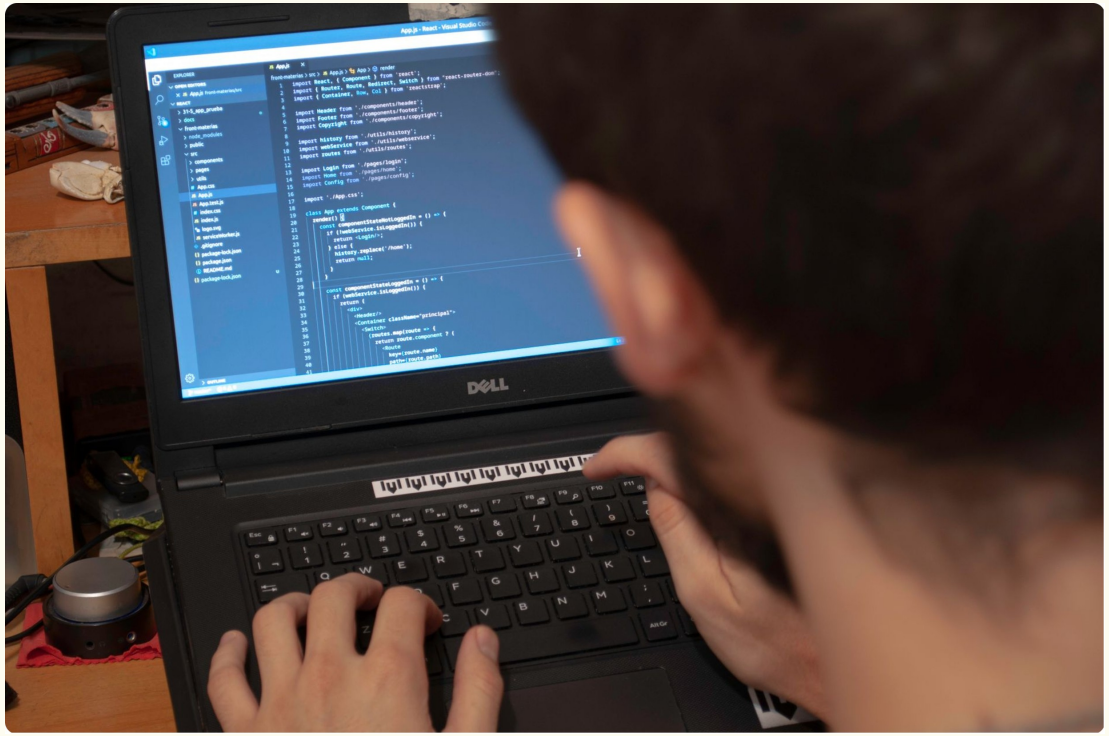
End-to-End Food Delivery Database System

DBMS Project — Division: 13 • B.Tech CSE-CSF

Team Members:
Miraya Ballav - 1262251072
Drishti Parashar - 1262250911
Saksham Jain - 1262250842
Ayush Manjrekar - 1262251824

A project focused on designing, implementing, and demonstrating a relational database for a food-delivery workflow.

Project Overview, Problem & Objectives



What is QuickBite?

QuickBite is an academic relational database system that models the food-ordering lifecycle: customers, restaurants, menus, orders, payments, delivery, and reviews — enabling structured storage, integrity, and queryable business insights.

Problem

Without a normalized DB design, platforms face duplication, inconsistent states (orders/payments), and inefficient multi-table retrievals for reporting and analytics.

Objectives

- Manage customers & addresses
- Model restaurants, categories, and food items
- Track orders, order items, payments, and deliveries
- Store reviews & ratings
- Demonstrate SQL features: joins, constraints, transactions

System Modules & End-to-End Flow



1. Customer Management

Profiles, authentication (planned), and addresses.



2. Restaurant Management

Restaurant meta, contact, location, cuisine, and rating.



3. Menu Management

Categories and food items with price and availability.



4. Order Management

Orders, statuses, timestamps, and order items.



5. Payment Management

Payments linked to orders, methods, amounts, and statuses.



6. Delivery Management

Delivery partners, delivery records, status, and times.



7. Review & Rating

Customer feedback tied to orders and restaurants.

Order Flow

Customer → Restaurant → Menu → Food Items → Order → Payment → Restaurant Processing → Delivery Partner → Delivered → Review

Modules link via primary and foreign keys to maintain referential integrity and enable JOIN-based reporting.

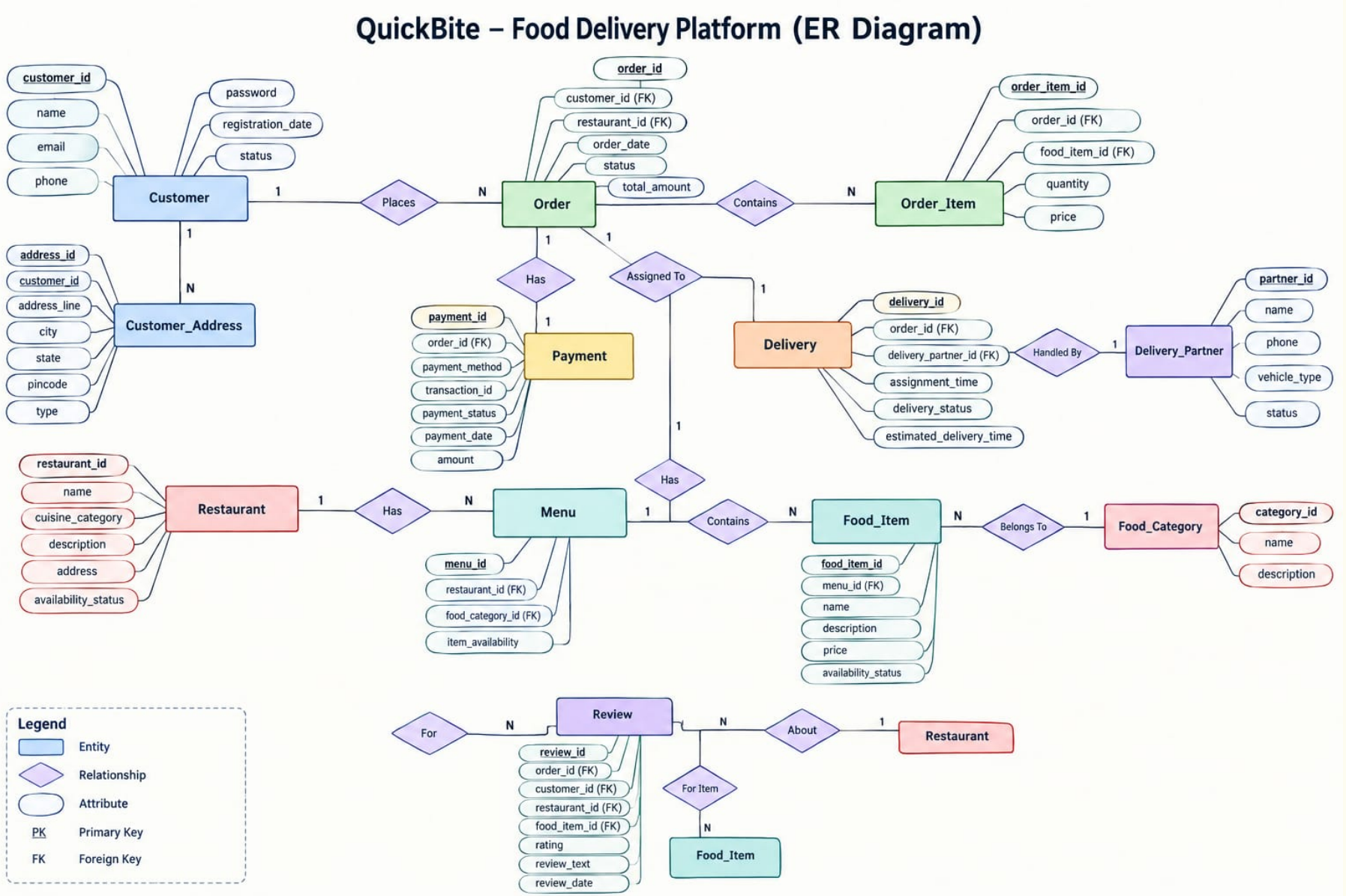
Database Tables & Key Attributes

Compact reference for the 11 core tables and important columns.

Table	Key Attributes (primary/important)
CUSTOMER	customer_id (PK), name, email, phone, password
ADDRESS	address_id (PK), customer_id (FK), address, city, pincode
RESTAURANT	restaurant_id (PK), name, location, cuisine, contact, rating
CATEGORY	category_id (PK), category_name
FOOD_ITEM	item_id (PK), restaurant_id (FK), category_id (FK), item_name, price, availability
ORDERS	order_id (PK), customer_id (FK), restaurant_id (FK), address_id (FK), order_date, total_amount, order_status
ORDER_ITEM	order_item_id (PK), order_id (FK), item_id (FK), quantity, price
PAYMENT	payment_id (PK), order_id (FK), payment_method, amount, payment_status, payment_date
DELIVERY_PARTNER	partner_id (PK), name, phone, vehicle_no
DELIVERY	delivery_id (PK), order_id (FK), partner_id (FK), delivery_status, delivery_time
REVIEW	review_id (PK), customer_id (FK), restaurant_id (FK), order_id (FK), rating, comment, review_date

Use PKs for uniqueness, FKs for relationships, and appropriate data types (INT, VARCHAR, DATE/TIME, DECIMAL, BOOLEAN).

ER Diagram & Relationships



Relational Schema

Requirements	Relational Algebra
All customer names	π name (CUSTOMER)
Active costumers	σ status='Active' (Customer)
Customer names + emails	π name, email (CUSTOMER)
Available restaurants	σ availability_status ='Available' (RESTAURANT)
Food items above ₹200	σ price>200 (FOOD_ITEM)
Delivered orders	σ status='Delivered' (ORDER)
Successful payments	σ payment_status = 'Successful' (PAYMENT)
Available food items	σ availability_status= 'Available' (FOOD_ITEM)

SQL, Keys & Constraints (MySQL)

SQL Commands Demonstrated

- **DDL:** CREATE / ALTER / DROP
- **DML:** INSERT / UPDATE / DELETE
- **DQL:** SELECT
- **DCL:** GRANT / REVOKE
- **TCL:** COMMIT / ROLLBACK / SAVEPOINT

Constraints

- PRIMARY KEY, FOREIGN KEY
- NOT NULL, UNIQUE
- CHECK, DEFAULT
- AUTO_INCREMENT

Representative SQL snippets

```
CREATE DATABASE QuickBite;
USE QuickBite;
```

```
SELECT item_name, price
FROM Food_Item
WHERE availability = TRUE;
```

```
SELECT SUM(amount) AS Total_Revenue
FROM Payment
WHERE payment_status = 'Paid';
```

Important SQL Queries (Examples & Purpose)

1. Customer Orders

```
SELECT c.name AS Customer,
    o.order_id,
    r.name AS Restaurant,
    o.total_amount,
    o.order_status
FROM Orders o
JOIN Customer c ON o.customer_id = c.customer_id
JOIN Restaurant r ON o.restaurant_id = r.restaurant_id;
```

Purpose: List orders with customer and restaurant context for support or reports.

2. Complete Order Details

```
SELECT o.order_id,
    c.name AS Customer,
    f.item_name,
    oi.quantity,
    oi.price,
    o.order_status
FROM Orders o
JOIN Customer c ON o.customer_id = c.customer_id
JOIN Order_Item oi ON o.order_id = oi.order_id
JOIN Food_Item f ON oi.item_id = f.item_id;
```

Purpose: Fetch itemized order contents for billing or order confirmation.

Other useful queries: display restaurants, available items, SUM() revenue, COUNT() orders per restaurant, AVG() ratings. JOINs + aggregates enable cross-table analytics.

Advanced DBMS Concepts Demonstrated



Views

Reusable virtual tables — e.g., Customer_Order_History combining customers, orders, and restaurants for quick reporting.



Triggers

Auto-update or validation when order/payment status changes (e.g., on payment success update order_status = 'Confirmed').



Stored Procedures

Encapsulate business logic: calculate order total, apply discounts, process order steps.



Transactions

Wrap multi-step operations (Insert Order → Insert Order_Items → Process Payment) with COMMIT/ROLLBACK to ensure atomicity.

Example flow: Place Order → Insert order & items → Process payment → COMMIT. On failure, ROLLBACK to maintain consistency.

Sample Data & End-to-End Order (Practical Example)



Transaction Snapshot

Customer: Drishti

Restaurant: Food Hub

Item	Price
Cheese Burger	₹150
Cold Coffee	₹100
Total	**₹250**

Payment: UPI — Paid

Delivery Partner: Amit

Delivery Status: Out for Delivery

This single order links records across Customer → Address → Restaurant → Food_Item → Orders → Order_Item → Payment → Delivery → Review, illustrating relational integrity and query paths.

Advantages, Future Scope & Conclusion

Advantages

- Organized data storage and reduced redundancy
- Improved consistency via constraints and transactions
- Efficient retrieval using JOINs and indexes
- Support for reporting, analytics, and integrity

Future Scope

- Authentication & role-based access (customers, restaurants)
- Real-time delivery tracking & notifications
- Payment gateway integration, refunds, cancellation handling
- Recommendation engine and mobile/web app integration

Conclusion

QuickBite models a realistic food-delivery ecosystem with clear emphasis on DBMS fundamentals: design, ER modeling, SQL, constraints, transactions, views, triggers, and stored procedures. The project is suitable for demonstration during evaluation and for extension into a full application later.

Thank You